

Edge Detection Operators

Edge detection can be understood by thinking of walking through a landscape in fog. Most of the time, the ground feels flat and consistent, but when you reach a cliff edge or a sharp drop, the change is sudden and noticeable. Edge detection works in a similar way: instead of physical height, it measures changes in pixel intensity. Smooth regions correspond to flat ground, while edges correspond to sudden “drops” or “jumps” in brightness.

Edge detection is a core image processing technique used to identify sharp changes in image intensity, which usually correspond to object boundaries or other important structural features in an image. By reducing an image to its most meaningful intensity changes, edge detection helps simplify visual data while preserving the shape information needed for later analysis.

This makes edge detection important in tasks such as image segmentation, object extraction, feature recognition, and general computer vision, where detecting boundaries is often the first step before higher-level interpretation. In simple terms, edges help show where one region ends and another begins, so they provide a compact but informative description of image structure.

The methods covered in this section can be grouped into two main families. **Gradient-based methods** detect edges using first-order derivatives, so they respond to strong local intensity changes, while **second-derivative methods** detect edges through zero-crossings after differentiation, often following smoothing. In this notes section, **Sobel** and **Prewitt** are first-order gradient operators, **Laplacian of Gaussian (LoG)** is a second-order zero-crossing method, and **Canny** is a multi-stage detector designed to balance noise reduction, localization, and edge clarity.

Real-World Example

A practical example of edge detection can be found in autonomous driving systems, where detecting road boundaries is critical. An input image from a front-facing camera is first converted to grayscale and optionally smoothed to reduce noise. The Canny edge detector is then applied to highlight strong intensity changes corresponding to lane markings and road edges.

The resulting edge map is further processed using techniques such as Hough Transform to identify straight lines, which represent lane boundaries. In this context, Sobel or Prewitt could be used for faster but less precise detection, while Canny is preferred due to its ability to produce clean and continuous edges under varying lighting conditions.

This demonstrates how edge detection acts as a preprocessing step, simplifying raw visual data into structured features that can be used for higher-level tasks such as navigation and decision-making.

Sobel Operator

Definition

The Sobel operator is a **3x3 gradient-based edge detector** that uses one kernel to measure horizontal intensity change and another to measure vertical intensity change. It is a first-order derivative method, so it finds edges by looking for strong changes in pixel values rather than zero-crossings. Compared with Prewitt, Sobel gives the center pixel more weight, which improves smoothing and makes it less sensitive to noise.

Kernel

The standard Sobel kernels are:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

The image is convolved with both kernels separately to produce horizontal and vertical gradient responses. The final edge strength is then obtained from these responses, usually by combining them into a magnitude image.

How it works

First, Sobel computes G_x by applying the horizontal kernel, which highlights vertical edges, and G_y by applying the vertical kernel, which highlights horizontal edges. The gradient magnitude is commonly calculated as $\sqrt{G_x^2 + G_y^2}$, or sometimes approximated by $[G_x] + [G_y]$ for faster computation. The edge direction is found using $\theta = \tan^{-1}(\frac{G_y}{G_x})$, which tells you the orientation of the intensity change. Because the center row and center column are weighted more heavily, Sobel smooths a little while differentiating, which helps reduce noise compared with a plain average-gradient mask.

Worked Example

Consider the following 3x3 grayscale image patch:

$$\begin{bmatrix} 10 & 10 & 10 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

This represents a strong horizontal intensity change.

Using the Sobel G_y kernel:

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

We compute the convolution:

$$(10 \times 1 + 10 \times 2 + 10 \times 1) + (0 \times 0 + 0 \times 0 + 0 \times 0) + (0 \times -1 + 0 \times -2 + 0 \times -1) = (10 + 20 + 10) + 0 + 0 = 40$$

So: $G_y = 40$

$$G_x \approx 0 \text{ (no vertical change)}$$

Gradient magnitude: $\sqrt{G_x^2 + G_y^2} = \sqrt{0 + 1600} = 40$

This high value confirms a strong edge.

For Edge direction:

$$\tan^{-1}(G_y / G_x) = \tan^{-1}(40 / 0) = \text{undefined}$$

This is one of the special cases, which means the edge line is perpendicular to the gradient direction. Therefore, in practice it is taken as a 90 degree gradient direction

Strengths

Sobel is simple, fast, and easy to implement, so it is a good choice for basic edge detection tasks. It works well for detecting strong outer boundaries and gives a useful estimate of edge orientation. Its built-in weighting also makes it more robust to noise than Prewitt in many cases.

Weaknesses

Sobel still remains sensitive to noise, especially in images with weak contrast or lots of texture, so it can produce broken or false edges. It is also limited as a simple local operator, so it does not give the clean thinning and edge linking that stronger detectors like Canny provide. In addition, Sobel is mainly best for basic horizontal and vertical structure, so it is less effective when edges are very fine, curved, or poorly defined.

Prewitt Operator

Definition

The Prewitt operator is a first-order gradient edge detector that is very similar to Sobel, but it uses **uniform weights** across the kernel instead of giving extra weight to the center pixel. It estimates horizontal and vertical intensity changes using two 3x3 masks. This makes it a simple tool for locating edges in basic image-processing tasks.

Kernel

The standard Prewitt kernels are:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

The image is convolved with both masks to estimate the gradients in the horizontal and vertical directions. These two responses are then combined to produce edge strength and orientation in the same general way as Sobel.

How it works

Prewitt detects edges by measuring changes in pixel intensity across neighboring rows and columns. The G_x mask highlights vertical edges, while the G_y mask highlights horizontal edges. After convolution, the gradient magnitude can be computed as $\sqrt{G_x^2 + G_y^2}$, and the edge direction can be estimated with $\tan^{-1}(\frac{G_y}{G_x})$. Because all kernel values are equally weighted, Prewitt behaves like a straightforward average-gradient detector rather than a smoothed one.

Strengths

Prewitt is easy to understand, easy to implement, and computationally light. It is useful when you want a fast first pass at edge detection without much complexity. For clean images with clear boundaries, it can give acceptable gradient estimates and orientation cues.

Weaknesses

Prewitt is usually noisier and less refined than Sobel because it does not emphasize the center pixel as strongly. That makes it weaker when image noise is high or when edge detail is subtle. In practice, it can miss fine structure or produce less stable edges than more robust operators.

Laplacian of Gaussian

Definition

The Laplacian of Gaussian, usually called **LoG**, is a **second-derivative edge detection method** that first smooths an image with a Gaussian filter and then applies the Laplacian operator. Unlike Sobel and Prewitt, which are first-order gradient methods, LoG belongs to the second-derivative family. Its main purpose is to detect locations where intensity changes sharply after smoothing.

Steps

LoG works in two main steps: **Gaussian smoothing** followed by **Laplacian filtering**. The Gaussian stage reduces noise before differentiation, which is important because the Laplacian is highly sensitive to noise and can otherwise respond strongly to random intensity fluctuations. In practice, the Gaussian and Laplacian are often combined into a single LoG kernel so that only one convolution is needed at run time.

How it works

After the image is smoothed, the Laplacian computes the second spatial derivative, which highlights regions of rapid intensity change. LoG does **not** locate edges by gradient maxima; instead, it finds edges at **zero crossings** in the second-derivative response. This means the detector looks for points where the filtered output changes sign, which typically occurs around object boundaries.

Worked Example

Consider the following 3×3 grayscale image patch:

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & 30 & 40 \\ 30 & 40 & 50 \end{bmatrix}$$

We apply a standard 3×3 Gaussian kernel:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Step 1: Multiply corresponding values

$$\begin{aligned}
 &(10 \times 1) + (20 \times 2) + (30 \times 1) \\
 &+(20 \times 2) + (30 \times 4) + (40 \times 2) \\
 &+(30 \times 1) + (40 \times 2) + (50 \times 1)
 \end{aligned}$$

Step 2: Compute each term

$$\begin{aligned}
 &= (10 + 40 + 30) + (40 + 120 + 80) + (30 + 80 + 50) \\
 &= 80 + 240 + 160 = 480
 \end{aligned}$$

Step 3: Apply normalization

$$\frac{480}{16} = 30$$

Final Result

The filtered center pixel becomes:

30

The original center value was **30**, and after Gaussian smoothing it remains **30**, but this is because the surrounding values are symmetrically distributed. In general, Gaussian filtering:

- reduces noise by averaging neighboring pixels
- gives more importance to closer pixels
- smooths sharp variations before edge detection

This is why Gaussian smoothing is applied before methods like **LoG** and **Canny**, to prevent noise from being falsely detected as edges.

Strengths

LoG can detect clear contours well and is useful when rotationally symmetric edge detection is needed, since the Laplacian has the same behavior in all directions. It is also conceptually neat because smoothing and edge localization are combined into one framework. For well-defined boundaries, LoG can highlight contour structure effectively.

Weaknesses

LoG remains noise-sensitive even with smoothing, because second-derivative methods naturally amplify small fluctuations more than first-order methods. It may also produce thick, double, or closed contours if the smoothing scale or thresholding is not chosen well. In addition, it does not directly provide edge orientation as naturally as gradient-based methods such as Sobel or Prewitt.

Canny Pipeline

Definition

The Canny edge detector was designed around three main goals: **good detection**, meaning real edges should be found with few false detections; **good localization**, meaning detected edges should lie as close as possible to the true boundary; and **single response per edge**, meaning one physical edge should not create multiple detections. Because of these goals, Canny is treated as a multi-stage and more optimal detector than simple single-mask operators. It is widely regarded as one of the strongest classical edge detectors in image processing.

Pipeline

The five main stages of the Canny pipeline are **Gaussian smoothing**, **gradient calculation**, **non-maximum suppression**, **double thresholding**, and **hysteresis edge tracking**. Gaussian smoothing reduces image noise before edge measurement. Gradient calculation estimates the strength and direction of intensity change, usually using Sobel-type derivatives. Non-maximum suppression thins broad edge responses down to narrow ridges by keeping only local maxima along the gradient direction. Double thresholding separates strong edge pixels from weak and likely noisy ones. Hysteresis edge tracking then keeps weak edges only when they are connected to strong edges, which helps preserve true contours while rejecting isolated noise responses.

How it works

In the first stage, Gaussian smoothing suppresses high-frequency noise so that small fluctuations are less likely to be mistaken for edges. Next, the image gradient is computed to measure how strongly intensity changes at each pixel and in which direction that change occurs. Non-maximum suppression compares a pixel with its neighbors along the gradient direction and removes it unless it is a local maximum, which is why the output edges become thin instead of thick. Double thresholding labels strong responses as reliable edges and weaker responses as possible edges. Finally, hysteresis keeps those weak responses only if they connect to strong edges, so true continuous boundaries survive while isolated weak noise is discarded.

Strengths

Canny is often treated as the standard classical edge detector because it balances **noise suppression** and **accurate localization** better than simpler operators such as Sobel or Prewitt. It usually produces thinner, cleaner, and more continuous edges because of non-maximum suppression and hysteresis thresholding. It also performs especially well in noisy conditions compared with many basic first-order methods.

Weaknesses

Canny is more computationally expensive than simpler operators because it uses several processing stages instead of one direct convolution. Its performance also depends on parameter choices such as the Gaussian scale and the two thresholds, so poor tuning can remove useful edges or keep too much noise. Even though it is usually the strongest classical option, it is still not perfect in very noisy or highly textured images.

Conclusion

Comparative Analysis of Edge Detectors

While all four methods aim to detect intensity changes, they differ significantly in performance, robustness, and output quality. Sobel and Prewitt, as first-order gradient operators, are computationally efficient and suitable for simple applications. However, they tend to produce thicker and noisier edges, particularly in images with high texture or low contrast.

The Laplacian of Gaussian improves edge localization through smoothing and second-derivative detection, but this comes at the cost of increased sensitivity to noise and the possibility of producing double edges due to zero-crossings. Unlike gradient-based methods, it does not directly provide edge orientation, which limits its usefulness in tasks requiring directional information.

Canny, by contrast, integrates multiple stages to address these limitations. Its use of non-maximum suppression ensures thin edges, while hysteresis thresholding improves continuity. As a result, it generally produces the most visually accurate and structured edge maps among the methods considered.

Implementation Notes

In most implementations, edge detection is applied after converting images to grayscale, as color information is not required for intensity-based gradients. Libraries such as OpenCV provide built-in functions for Sobel, Laplacian, and Canny, allowing efficient computation even on large images.

However, understanding the underlying convolution process remains important, as it explains how parameter choices directly affect the output.

Failure Cases and Limitations

Each method exhibits specific failure modes under different conditions. In low-contrast images, gradient-based operators such as Sobel and Prewitt may fail to detect edges due to weak intensity differences. In highly textured regions, they may instead produce excessive edge responses, making the output difficult to interpret.

LoG can struggle in noisy environments, as second-order derivatives amplify small fluctuations even after smoothing. This can lead to false zero-crossings and fragmented contours. Additionally, lighting variations and shadows can introduce edges that do not correspond to actual object boundaries.

Even Canny, despite its strengths, is not immune to these issues. In highly complex scenes, it may still detect unwanted edges or fail to capture very subtle boundaries if parameters are not well tuned.

Evaluation

The comparison shows that no single edge detection method is universally optimal. Simpler operators such as Sobel and Prewitt are useful for fast and basic applications, while LoG provides a more mathematically structured approach to edge detection through zero-crossings. However, the Canny detector consistently offers the best balance between noise reduction, edge localization, and continuity.

The choice of method ultimately depends on the specific requirements of the task, including computational constraints, image quality, and the level of detail required. This highlights the importance of understanding both the theoretical foundations and practical limitations of each approach.

Quick Reference

Method	Type	Key Idea	Strengths	Weaknesses	When to Use
Sobel	First-order (gradient)	Computes intensity change using weighted kernels	Simple, fast, gives edge direction, slightly noise-resistant	Produces thick edges, still noise-sensitive	Basic edge detection, real-time applications
Prewitt	First-order (gradient)	Computes intensity change using uniform kernels	Very simple, computationally light	More noise-sensitive than Sobel, less accurate	Quick approximation where speed matters
LoG	Second-order (zero-crossing)	Smooths image then detects zero-crossings in second derivative	Good contour detection, rotation invariant	Sensitive to noise, can produce double edges, no orientation	Detecting closed contours or symmetric structures

Canny	Multi-stage	Combines smoothing, gradients, thinning, and thresholding	Thin, clean, continuous edges, robust to noise	Computationally heavier, needs parameter tuning	High-quality edge detection, real-world applications
Gaussian Filter	Smoothing (preprocessing)	Weighted averaging of neighboring pixels	Reduces noise, stabilises edge detection	Blurs fine details if overused	Always before edge detection (especially Canny, LoG)

Key Equations

Gradient magnitude:

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}$$

Gradient direction:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

Gaussian kernel (3×3 example):

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Key Takeaways

- Edge detection = **finding rapid intensity change**
- First-order methods → detect **where change is strong**
- Second-order methods → detect **where change flips**
- Canny → combines multiple steps for **best overall result**
- Gaussian smoothing → **essential to reduce noise before detection**

Key Papers

1) Study and Comparison of Various Image Edge Detection Techniques

Full citation:

R. Maini and H. Aggarwal, "Study and Comparison of Various Image Edge Detection Techniques," *International Journal of Image Processing*, vol. 3, no. 1, pp. 1–12, 2011.

Direct URL:

<https://www.cscjournals.org/manuscript/Journals/IJIP/Volume3/Issue1/IJIP-15.pdf>

Summary:

This paper compares common classical edge detectors, especially Sobel, Prewitt, Roberts, Laplacian of Gaussian, and Canny, using MATLAB-based experiments. It explains the two main families of edge detection, why gradient-based methods rely on first derivatives, and why Laplacian-based methods use zero-crossings. The paper also discusses smoothing, localization, false edges, and the trade-off between accuracy and computational cost. Its results show that Canny generally gives the best visual performance, particularly in noisy images, although it is more expensive computationally.

Relevance:

This is one of the best sources for your notes because it directly supports all four operators in your section: Sobel, Prewitt, LoG, and Canny. It is also especially useful for the strengths and weaknesses wording in a student-note format.

2) Comparative Study Among Sobel, Prewitt and Canny Edge Detection Operators Used in Image Processing

Full citation:

A. S. Ahmed, "Comparative Study Among Sobel, Prewitt and Canny Edge Detection Operators Used in Image Processing," *Journal of Theoretical and Applied Information Technology*, vol. 96, no. 19, pp. 6517–6525, 2018.

Direct URL:

<http://www.jatit.org/volumes/Vol96No19/20Vol96No19.pdf>

Summary:

This paper compares Sobel, Prewitt, and Canny using performance measures such as execution time, MSE, and PSNR. It explains the gradient-based logic behind Sobel and Prewitt, then describes Canny as a multi-stage detector with smoothing, gradient calculation, non-maximum suppression, double thresholding, and hysteresis. The paper's experiments suggest that Canny often gives better edge detection quality, but at a higher computational cost. It also reinforces the idea that Prewitt is lighter and simpler while Sobel offers slightly better noise suppression.

Relevance:

This paper is highly relevant because it gives compact operator descriptions that match your study-notes format and supports your comparison language for Sobel, Prewitt, and Canny. It is especially useful if you want a short, comparison-friendly reference with practical metrics.

3) A Comprehensive Analysis of Image Edge Detection Techniques

Full citation:

M. A. Ansari, D. Kurchaniya, and M. Dixit, "A Comprehensive Analysis of Image Edge Detection Techniques," *International Journal of Multimedia and Ubiquitous Engineering*, vol. 12, no. 11, pp. 1–12, 2017.

Direct URL:

https://gvpress.com/journals/IJMUE/vol12_no11/1.pdf

Summary:

This paper surveys several standard edge detectors, including Roberts, Prewitt, Sobel, LoG, and Canny, and evaluates them using PSNR and MSE on test images with and without noise. It gives short algorithm descriptions, explains the basic kernels, and discusses the advantages and disadvantages of each method. The results indicate that Canny performs best overall in noisy settings, while LoG is more sensitive to noise and simpler gradient-based methods are easier to implement. The paper is written in a straightforward comparative style, making it easy to extract revision-friendly points.

Relevance:

This is a strong fit for your notes because it supports the exact four-part structure of your section while also giving you comparison language for strengths and weaknesses. It is especially useful for LoG and Canny, since it clearly contrasts second-derivative sensitivity with Canny's better practical performance.

4) Edge Detection Techniques—An Overview

Full citation:

D. Ziou and S. Tabbone, "Edge Detection Techniques—An Overview," in *Pattern Recognition and Image Analysis*, 1998, pp. 537–559.

Direct URL:

<https://inria.hal.science/inria-00098446v1/document>

Summary:

This overview paper explains edge detection from a conceptual and theoretical perspective, focusing on edge definitions, detector properties, smoothing, differentiation, labeling, and multi-scale approaches. It distinguishes between step edges, line edges, and junctions, and explains how first- and second-derivative methods relate to these structures. The paper also discusses regularization, noise reduction, and the trade-off between smoothing and localization, which are central ideas behind Canny and LoG. Because it is broader than the other three papers, it helps ground the notes in the theory behind classical detectors.

Relevance:

This is the best background paper for your introduction and for the theory behind the pipeline logic used by Sobel, LoG, and Canny. It is especially helpful if you want to justify why edge detection is divided into gradient-based and second-derivative families.